

Cheat-Sheet

Quick-Reference zum Projekt – Datenanwendung mit Python

Hinweis

Dieses Dokument ist als **Nachschlagewerk** gedacht, nicht als Tutorial. Wenn Sie eine Bibliothek zum ersten Mal nutzen, lesen Sie zuerst die offizielle Dokumentation oder einen Einführungstext. Dieses Cheat-Sheet hilft Ihnen ab dem Punkt, an dem Sie wissen *wonach* Sie suchen.

Setup und Projektstart

Code-Beispiel: Virtuelle Umgebung anlegen und Bibliotheken installieren

```
1 # Verzeichnis fuer das Projekt erstellen
2 mkdir mein_projekt && cd mein_projekt
3
4 # Virtuelle Umgebung anlegen
5 python -m venv .venv
6
7 # Aktivieren (Linux/Mac)
8 source .venv/bin/activate
9
10 # Aktivieren (Windows PowerShell)
11 .venv\Scripts\Activate.ps1
12
13 # Aktivieren (Windows CMD)
14 .venv\Scripts\activate.bat
15
16 # Bibliotheken installieren (Basis)
17 pip install streamlit requests pandas python-dotenv openpyxl
18
19 # Aktuelle Bibliotheken in requirements.txt einfrieren
20 pip freeze > requirements.txt
21
22 # Spaeter: aus requirements.txt installieren
23 pip install -r requirements.txt
```

Code-Beispiel: Sinnvolle .gitignore

```
1 .venv/
2 __pycache__/
3 *.pyc
4 .env
5 *.db
6 *.sqlite
```

```
7 daten/*.xlsx
8 .idea/
9 .vscode/
```

Konfiguration und Geheimnisse

Code-Beispiel: .env-Datei lesen

```
1 # datei: .env (NICHT in Git!)
2 # API_KEY=mein_geheimer_schlüssel
3 # DATENBANK_PFAD=./daten/projekt.db
4
5 # datei: konfiguration.py
6 from dotenv import load_dotenv
7 import os
8
9 load_dotenv()
10
11 API_KEY = os.getenv("API_KEY", "") # leer = nicht gesetzt
12 DB_PFAD = os.getenv("DATENBANK_PFAD", "./projekt.db")
13
14 if not API_KEY:
15     raise RuntimeError("API_KEY nicht gesetzt. Bitte .env pruefen.")
```

Streamlit – Frontend in Minuten

Code-Beispiel: Grundgeruest einer Streamlit-Anwendung

```
1 import streamlit as st
2
3 st.set_page_config(page_title="Mein Tool", layout="wide")
4 st.title("Mein Tool")
5
6 # Sidebar fuer Einstellungen
7 with st.sidebar:
8     st.header("Einstellungen")
9     schwelle = st.slider("Schwellwert", 0, 100, 50)
10
11 # Eingaben
12 stadt = st.text_input("Stadtname")
13 absenden = st.button("Abrufen")
14
15 # Reaktion auf Klick
16 if absenden and stadt:
17     with st.spinner("Daten werden geladen..."):
```

```
18     # ... API-Aufruf, Verarbeitung
19     ergebnis = {"stadt": stadt, "wert": 42}
20     st.success(f"Erfolgreich: {ergebnis['stadt']}")
21     st.json(ergebnis)
```

Code-Beispiel: Streamlit-Widgets im Ueberblick

```
1 import streamlit as st
2 import pandas as pd
3
4 # Eingabe
5 name      = st.text_input("Name")
6 zahl      = st.number_input("Wert", min_value=0, max_value=100,
7                             value=10)
8 auswahl   = st.selectbox("Variante", ["A", "B"])
9 mehrfach  = st.multiselect("Staedte", ["Berlin", "Hamburg", "
10                                Muenchen"])
11 datum     = st.date_input("Datum")
12 ja_nein   = st.checkbox("Erweiterte Ausgabe")
13
14 # Ausgabe
15 st.write("Einfache Ausgabe")
16 st.markdown("**Fett**", _kursiv_, 'Code')
17 st.dataframe(pd.DataFrame({"a": [1, 2], "b": [3, 4]}))
18 st.line_chart({"reihe": [1, 4, 3, 5]})
19 st.metric("Temperatur", "5.2 C", "+0.7")
20
21 # Layout
22 spalte1, spalte2 = st.columns(2)
23 with spalte1:
24     st.write("Links")
25 with spalte2:
26     st.write("Rechts")
```

Hinweis

Streamlit-Eigenheit: Bei jeder Eingabe wird das gesamte Skript erneut von oben nach unten ausgeführt. Wer das nicht beachtet, ist über mehrfache API-Aufrufe verwirrt. Lösungen: `@st.cache_data` für teure Berechnungen, `st.session_state` für Zustand über mehrere Reloads hinweg.

SQLite – Datenbank in einer Datei

Code-Beispiel: Verbindung – Tabelle – Daten

```
1 import sqlite3
2 from contextlib import closing
3
```

```
4 DB_PFAD = "daten/projekt.db"
5
6 def initialisiere_datenbank():
7     """Legt die Tabellen an, falls sie noch nicht existieren."""
8     with closing(sqlite3.connect(DB_PFAD)) as verbindung:
9         verbindung.execute("""
10             CREATE TABLE IF NOT EXISTS staedte (
11                 id INTEGER PRIMARY KEY AUTOINCREMENT,
12                 name TEXT NOT NULL UNIQUE,
13                 land TEXT,
14                 breitengrad REAL,
15                 laengengrad REAL,
16                 angelegt_am TEXT DEFAULT CURRENT_TIMESTAMP
17             )
18         """)
19         verbindung.commit()
20
21 def stadt_einfuegen(name, land, lat, lon):
22     """Speichert eine neue Stadt."""
23     with closing(sqlite3.connect(DB_PFAD)) as verbindung:
24         verbindung.execute(
25             "INSERT INTO staedte (name, land, breitengrad,
26                 laengengrad) "
27             "VALUES (?, ?, ?, ?)",
28             (name, land, lat, lon)
29         )
30         verbindung.commit()
31
32 def alle_staedte():
33     """Liefert alle Staedte als Liste von Dicts."""
34     with closing(sqlite3.connect(DB_PFAD)) as verbindung:
35         verbindung.row_factory = sqlite3.Row
36         return [dict(zeile) for zeile in
37             verbindung.execute("SELECT * FROM staedte")]
```

Wichtig!

Niemals Werte per String-Verkettung in SQL einbauen.

Falsch: `f"SELECT * FROM x WHERE name = '{name}'"` – öffnet SQL-Injection.

Richtig: `"SELECT * FROM x WHERE name = ?"` mit Tupel `(name,)` als zweites Argument.

requests – HTTP und APIs

Code-Beispiel: Robuster API-Aufruf mit Fehlerbehandlung

```
1 import requests
2
3 API_URL = "https://api.open-meteo.com/v1/forecast"
```

```
4
5 def wetter_abrufen(breitengrad, laengengrad):
6     """Ruft aktuelle Temperatur ab. Liefert dict oder loest aus."""
7     params = {
8         "latitude": breitengrad,
9         "longitude": laengengrad,
10        "current": "temperature_2m,relative_humidity_2m"
11    }
12    try:
13        antwort = requests.get(API_URL, params=params, timeout=5)
14        antwort.raise_for_status() # 4xx/5xx -> Exception
15        daten = antwort.json()
16        return daten["current"]
17    except requests.exceptions.Timeout:
18        raise RuntimeError("Wetter-API antwortet nicht (Timeout).")
19    except requests.exceptions.ConnectionError:
20        raise RuntimeError("Keine Internetverbindung.")
21    except requests.exceptions.HTTPError as fehler:
22        raise RuntimeError(f"API-Fehler: {fehler.response.status_code}")
23    except (KeyError, ValueError):
24        raise RuntimeError("Unerwartete Antwort der API.")
```

Code-Beispiel: API mit API-Key (Header oder Parameter)

```
1 import os, requests
2
3 API_KEY = os.getenv("API_KEY")
4
5 # Variante 1: Schluessel als Query-Parameter
6 antwort = requests.get(
7     "https://api.beispiel.de/v1/daten",
8     params={"q": "Berlin", "appid": API_KEY},
9     timeout=5
10 )
11
12 # Variante 2: Schluessel als Header (oft modern, sicherer)
13 antwort = requests.get(
14     "https://api.beispiel.de/v1/daten",
15     headers={"Authorization": f"Bearer {API_KEY}"},
16     timeout=5
17 )
```

pandas – Datenverarbeitung

Code-Beispiel: DataFrame anlegen und untersuchen

```
1 import pandas as pd
2
3 # Aus Liste von Dicts
4 daten = [
5     {"stadt": "Berlin", "tag": "2026-01-15", "temp": 4.5},
6     {"stadt": "Hamburg", "tag": "2026-01-15", "temp": 5.1},
7     {"stadt": "Berlin", "tag": "2026-01-16", "temp": 3.8},
8 ]
9 df = pd.DataFrame(daten)
10
11 # Aus SQLite direkt
12 import sqlite3
13 with sqlite3.connect("projekt.db") as conn:
14     df = pd.read_sql("SELECT * FROM staedte", conn)
15
16 # Aus CSV
17 df = pd.read_csv("daten.csv")
18
19 # Aus Excel
20 df = pd.read_excel("daten.xlsx", sheet_name="Tabelle1")
21
22 # Erster Ueberblick
23 df.head()           # erste 5 Zeilen
24 df.info()           # Spalten, Datentypen, Anzahl
25 df.describe()       # Statistiken pro numerischer Spalte
26 df.shape            # (Zeilen, Spalten)
27 df.columns          # Spaltennamen
```

Code-Beispiel: Filtern – Sortieren – Aggregieren

```
1 # Spalte auswaehlen
2 temps = df["temp"]
3
4 # Mehrere Spalten
5 sub = df[["stadt", "temp"]]
6
7 # Zeilen filtern
8 warm = df[df["temp"] > 4]
9 berlin = df[df["stadt"] == "Berlin"]
10 warm_in_berlin = df[(df["stadt"] == "Berlin") & (df["temp"] > 4)]
11
12 # Sortieren
13 df_sortiert = df.sort_values("temp", ascending=False)
14
15 # Gruppieren und aggregieren
16 durchschnitt = df.groupby("stadt")["temp"].mean()
17 zusammenfassung = df.groupby("stadt").agg(
18     mittel=("temp", "mean"),
19     minimum=("temp", "min"),
20     maximum=("temp", "max"),
21     anzahl=("temp", "count"),
```

```
22 )
```

Code-Beispiel: Berechnete Spalten und Exporte

```
1 # Neue Spalte
2 df["temp_f"] = df["temp"] * 9 / 5 + 32
3
4 # Bedingte Spalte
5 df["kategorie"] = df["temp"].apply(
6     lambda t: "kalt" if t < 5 else "mild" if t < 15 else "warm"
7 )
8
9 # Datum
10 df["tag"] = pd.to_datetime(df["tag"])
11 df["wochentag"] = df["tag"].dt.day_name()
12
13 # Exporte
14 df.to_csv("export.csv", index=False)
15 df.to_excel("export.xlsx", index=False)
16 with sqlite3.connect("projekt.db") as conn:
17     df.to_sql("wetter_historie", conn, if_exists="append", index=False)
```

Excel mit openpyxl (Alternative zu SQLite)

Code-Beispiel: Excel-Datei schreiben und lesen

```
1 from openpyxl import Workbook, load_workbook
2 from openpyxl.styles import Font, PatternFill
3
4 # Schreiben
5 wb = Workbook()
6 ws = wb.active
7 ws.title = "Daten"
8
9 # Kopfzeile
10 header = ["Datum", "Stadt", "Temperatur"]
11 ws.append(header)
12 for zelle in ws[1]:
13     zelle.font = Font(bold=True, color="FFFFFF")
14     zelle.fill = PatternFill("solid", fgColor="2980B9")
15
16 # Daten
17 ws.append(["2026-01-15", "Berlin", 4.5])
18 ws.append(["2026-01-15", "Hamburg", 5.1])
19
20 # Spaltenbreite
21 ws.column_dimensions["A"].width = 12
22 ws.column_dimensions["B"].width = 14
```

```
23
24 wb.save("export.xlsx")
25
26 # Lesen
27 wb = load_workbook("export.xlsx")
28 ws = wb["Daten"]
29 for zeile in ws.iter_rows(min_row=2, values_only=True):
30     print(zeile)
```

Logging statt print

Code-Beispiel: Einfaches und brauchbares Logging

```
1 import logging
2
3 logging.basicConfig(
4     level=logging.INFO,
5     format="%(asctime)s | %(levelname)-8s | %(name)s | %(message)s"
6     ,
7     datefmt="%Y-%m-%d %H:%M:%S",
8 )
9 log = logging.getLogger(__name__)
10
11 log.debug("Detail fuer Entwickler")
12 log.info("Normale Ablaufinfo")
13 log.warning("Ungewoehnlich, aber nicht kritisch")
14 log.error("Etwas ist schiefgegangen")
15
16 try:
17     result = 1 / 0
18 except ZeroDivisionError:
19     log.exception("Unerwarteter Fehler") # inkl. Traceback
```

Typische Fehlerquellen

Wichtig!

API-Antwort enthält das erwartete Feld nicht. Lösung: Vor dem Zugriff prüfen oder `KeyError` abfangen.

Wichtig!

Datenbank-Datei wird vom Programm gesperrt. Lösung: Verbindungen sauber schließen (`with closing(...)`).

Wichtig!

Streamlit ruft Funktionen mehrfach auf. Lösung: Berechnungen mit `@st.cache_data` markieren, oder Logik nur in `if button:` ausführen.

Wichtig!

API-Schlüssel wurde nach Git-Commit veröffentlicht. Lösung: `.env` vor dem ersten Commit in `.gitignore` eintragen. Falls bereits geschehen: Schlüssel sofort beim Anbieter widerrufen.

Wichtig!

pandas wirft SettingWithCopyWarning. Hintergrund: Versuche, einen DataFrame-Ausschnitt zu ändern. Lösung: Mit `.copy()` explizit kopieren oder mit `.loc[]` arbeiten.

Sinnvolle Suchbegriffe für Eigenrecherche

- `streamlit cache_data session_state` – für Performance und Zustand
- `sqlite3 row_factory` – für Dict-artigen Zugriff auf Zeilen
- `python-dotenv quickstart` – für die `.env`-Verarbeitung
- `pandas groupby agg multiple columns` – für Mehrfach-Aggregationen
- `requests timeout retry` – für robuste API-Aufrufe
- `openpyxl conditional formatting` – für Excel-Kür-Features
- `flask jinja templates` – falls Sie Flask statt Streamlit nutzen
- `logging python howto` – für strukturiertes Logging

Wenn nichts mehr geht

Praxistipp**Die Reihenfolge der Selbsthilfe:**

1. Fehlermeldung wirklich lesen. Vollständig. Auch die untere Hälfte.
2. Eine `print()`- oder `log.info()`-Anweisung an die fragliche Stelle setzen und prüfen: Was steht da wirklich drin?
3. In der offiziellen Dokumentation der Bibliothek nach dem Konzept suchen.
4. Eine gezielte Suche mit dem Kern der Fehlermeldung.

5. Eine konkrete Frage formulieren – entweder an eine andere Person oder an ein KI-Werkzeug.
6. Dabei: Mindestbeispiel zeigen, nicht das ganze Projekt.

*Erik Schwob – Robotron Bildungszentrum Halle
Cheat-Sheet zum Projekt 1, Version vom June 2, 2026*